pyaces Developer Manual

Structure, build, extension and testing of `aces2py`, the f2py extension module that drives ACES II from Python

Repository	`ACES_PROGRAM/pyaces` (GitHub youngchoonpark/ACES, private)
Reference commit	`f97a9759` (master, 2026-10-23)
Written	2026-10-23
Verified	pyaces suite 51/51 PASS on the local workstation (oneAPI 2024.0) and the ccsd server (oneAPI 2023.2)
Link audit	296 collision-baseline entries, 0 COMMON shadows, identical on both machines

This manual was written from the source code (`pyaces/build.sh`, `a2_pyf/*.F`, the generated `Main/gen/aces2pymodule.c`, the test scripts) and from inspecting the built shared object. Line numbers refer to the reference commit.

Contents

1. Overview
2. Directory layout
3. Build environment
4. Build pipeline (`build.sh`)
5. Linking into one .so: symbol collisions and their fixes
6. The link-resolution audit (`collision_audit.py`)
7. The `aces2py` API
8. Running calculations
9. The test suite
10. Extending pyaces: new entry points and new tests
11. Known issues (with fix history)
12. Development history and server deployment

Appendix A. Legacy files · Appendix B. Command summary

1. Overview

pyaces lets Python call the computational modules of the Fortran quantum-chemistry program ACES II (`ACES_PROGRAM/aces2`) directly. Classic ACES II builds one executable per module (`xjoda`, `xvscf`, `xvcc`, ...); a driver runs them in turn and they exchange state through files on disk (JOBARC, MOINTS, ...). pyaces links those modules into **a single shared library**, `aces2py.cpython-312-x86_64-linux-gnu.so`, and puts a Python wrapper generated by numpy's f2py on top.

Python: `import aces2py; aces2py.Init(); aces2py.Runcc(0); aces2py.Dgetrecpy("TOTENERG", e)`

▼ f2py C wrapper (`Main/gen/aces2pymodule.c`)

pyaces Fortran shims: `Main/init.F`, `a2_pyf/run*.F`, `a2_pyf/*recpy.F`, `*lstpy.F`, `aux_pyaces/*.F`

▼ static link (`-Wl,--start-group` ... about 107 archives)

aces2 archives: `aces2/lib/lib*.a` (joda, vmol, vscf, vtran, intprc, vcc, lambda, vee, props, intgrt, dirmp2, ...), some replaced by patched copies in `build/patched_libs`

▼

Intel MKL ILP64 (sequential) and the Intel Fortran runtime

1.1 Key facts

- **The aces2 sources are never touched.** pyaces links aces2's `.a` files as they are. Only for archives with collisions does it make temporary copies and rename symbols in them with `objcopy`. The aces2 tree is shared with the classic driver and its test suite.
- **All integers are 64-bit.** aces2 and pyaces are compiled with `-i8`, and MKL is linked in its ILP64 flavour.
- **One calculation per process.** aces2's COMMON-block state assumes a single use; calling Vtran/Vcc twice in one process has been seen to converge silently to a wrong answer.
- **The main risk is at link time.** Routines that share a name but differ in implementation were harmless when every executable was linked on its own. In one .so they collide, and the linker silently picks one. The safeguards for this are the core of chapters 5 and 6.
- **Tests reuse the aces2 fast suite.** Of its 125 cases, the 51 that pyaces can run (SCF 9, CC 31, EOM 8, KS 3) are executed and their JOBARC energies compared with the reference values to within 1×10^{-6} E_h.

1.2 How to read this manual

If you are new to pyaces, read chapter 2 (which files are live), then chapter 4 (the build), then chapters 7–8 (API and running). Before exposing a new Fortran routine or linking another aces2 module, read chapters 5, 6 and 10. Chapter 11 lists the defects still present in the code.

2. Directory layout

`pyaces/` still carries much of the original 2023 acespy, so it matters which files the current build actually uses. live marks files used by the build or tests, generated marks build output (gitignored), and legacy marks files nothing uses.

Path	Status	Contents
`build.sh`	live	The whole build (about 510 lines): f2py wrapper generation, C/Fortran compilation, patched archives, link, import test, link audit. The history of every collision fix is recorded in its comments.
`Main/aces2.pyf`	live	The f2py signature file. It sets the module name `aces2py`. **The final signatures, however, come from the Fortran sources** (section 7.1).
`Main/init.F`, `Main/aces2_main.F`	live	The `Init()` entry point. Joda reads ZMAT, then `aces_init` sets up core memory, flags and lists.
`Main/*.com`	live	Header copies (with explicit `integer*8`) that init.F picks up through quoted `#include`. They differ from aces2's headers (chapter 11).
`a2_pyf/*.F`	live	Wrappers and drivers exposed to Python: `run*.F`, `*recpy.F`, `*lstpy.F`, `*allpy.F`, `get1ehpy/get2ehpy.F`, `acesja*.F`, plus the internal `a2_get*/a2_put*.F`. `rungrads`, `run1props` and `run` are excluded from the build.
`aux_pyaces/*.F`	live	`init_legacies` (initialises legacy COMMON blocks), `get1ents` / `get2eints` (read AO integrals from the IIII file), `sym_expand`.
`tools/collision_audit.py`	live	The link-resolution audit (chapter 6), with `collision_baseline.txt` (296 entries) and `common_shadow_baseline.txt` (0).
`tools/compat_bin/ifort`	live	A two-line shim, `exec ifx "$@"`, so that `ifort` resolves to ifx on oneAPI releases without ifort.
`pyaces_run_case.py`	live	Runs one case in its own process and prints the result as marker lines (chapter 8).
`pyaces_suite_runner.py`	live	Runs the 51 cases and grades each one PASS/MISMATCH/... (chapter 9).
`classify_suite.py`, `suite_classification.tsv`	live	The table that classifies the aces2 fast-suite cases by category, entry point and reference value, and the script that first generated it. **The TSV contains hand-edited rows** (section 9.4).
`docs/`	live	This manual (HTML source and PDF).
`Main/gen/`	generated	f2py output: `aces2pymodule.c`, `aces2py-f2pywrappers.f`.
`Main/build/`	generated	All `.o` files and the final `aces2py.cpython-312-x86_64-linux-gnu.so`.
`build/patched_libs/`	generated	Copies of aces2 archives with renamed symbols (about 37). Wiped and rebuilt on every build.

`f2py_gen.log`	generated	The f2py wrapper-generation log. Read it to see how the wrappers were built.
`Fxx/`, `include/`, `a2_pyf/include_no/`, `Main/Archive/`, `a2_pyf/Archive/`, `Main/Pilot_tests/`, `Exp/`, `Python/`, `rename.py`, `README`, `frequently_asked_questions`	legacy	The acespy-era per-module shared-library build, old headers, test stubs and an f2py tutorial. Unrelated to the current build (Appendix A).

> **Lower-case `.f` files**: `a2_pyf/*.f`, `Main/init.f` and similar are old preprocessed copies. The build only uses `*.F`, so edit the `.F` files.

3. Build environment

3.1 Required tools

Item	Requirement
Fortran	Intel `ifx` (verified with oneAPI 2023.2 and 2024.0). The name `ifort` is provided by the `tools/compat_bin` shim.
C	`mpicc`, to compile the f2py C wrapper and `fortranobject.c` (it wraps gcc on the local workstation).
Python	**3.12** with numpy (including f2py). The output file name is hard-coded to `cpython-312`, so other versions cannot import it.
BLAS/LAPACK	Intel MKL; `MKLROOT` must be set. Linked as ILP64, sequential.
binutils	`ar`, `nm`, `objcopy`
Environment modules	The `module` command. `pyaces/build.sh` runs `module purge` and loads its own set.
Prerequisite	aces2 must be built first, so that `aces2/lib/lib*.a` exists.

3.2 configure and the module set

The top-level `./configure` finds a toolchain and writes `build_config.sh`. The top-level `build.sh` exports the `ACES_PROGRAM_MODULES` recorded there, and `pyaces/build.sh` reloads exactly that set after `module purge`. If the variable is empty it falls back to the HiPerGator set (`intel/2025.1.0 openmpi/5.0.7 python/3.12`).

```
# ccsd server
./configure --modules="intel/oneapi_2023.2.0 python/3.12.3" --np=4
# local workstation: build_config.sh has ACES_PROGRAM_MODULES="intel-ompi/2024.0"
```

> **Caution**: include a Python module in the set. If `module purge` unloads the Python that has numpy, the f2py step fails.

3.3 Locale

`pyaces/build.sh` forces `export LC_ALL=C` (line 21). The globs and `sort` calls in the script set archive-member order and `-l` order, and that order decides which copy of a duplicated routine gets linked. For the same reason the aces2 makefile wraps its source lists in `$(sort …)`: GNU Make 4.2 does not sort wildcard results, and 4.3 sorts them by locale.

3.4 Build commands

```
./build.sh aces2 pyaces          # top level: aces2, then pyaces
./build.sh pyaces                # only pyaces, when aces2 is already built
bash pyaces/build.sh             # run directly (set ACES_PROGRAM_MODULES yourself)
```

The log is `build_logs/unified_build_pyaces.log`. On the shared local workstation, stay inside the allocated cores: `taskset -c 20-23 flock -s /tmp/qtoy-bench.lock ./build.sh pyaces`. On the ccsd

server, build only through PBS (`build_logs/build_ccsd.pbs`).

> **After changing compiler flags**: make does not track flag changes. After changing aces2 flags, remove its build output with `git clean -fdXq aces2` and rebuild. pyaces wipes and recreates `gen/` and `build/` on every build.

4. Build pipeline (`build.sh`)

4.0 Setup (lines 1–38)

- Prepend `tools/compat_bin` to PATH, so `ifort` runs ifx.
- `module purge`, then `module load $ACES_PROGRAM_MODULES`. On failure, exit with a hint to rerun configure.
- Exit if `MKLROOT` is empty. `MKL=$MKLROOT/lib/intel64` (or `$MKLROOT/lib`).
- `export LC_ALL=C`.
- Ask the running python3 for `NUMPY_INC`, `PY_INC` and `F2PY_SRC` (numpy's `f2py/src`).
- `cd Main; rm -rf gen build; mkdir -p build`.

4.1 Source lists

`PYF_SOURCES` (lines 47–52) lists only the sources that *define* routines declared in the `.pyf`. Given only the `.pyf`, f2py generates empty stub wrappers; given every source, it auto-wraps every subroutine it finds and then trips over case mangling in the all-caps F77 names. Hence the hand-picked list.

`SOURCES` (lines 54–64) is every Fortran file to compile: `a2_pyf/*.F` except `rungrads`, `run1props` and `run`, plus `Main/init.F`, `Main/aces2_main.F` and `aux_pyaces/*.F`. So `a2_get*/a2_put*`, `getlstpy_dbg`, `get1epy` and `returnpy` are linked but not exposed to Python.

4.2 Steps

Step	Command / action	Output
1. Generate wrappers	`python3 -m numpy.f2py aces2.pyf $PYF_SOURCES -m aces2py --build-dir gen` (generate only, no `-c`). Then a `sed` lower-cases the first argument of every `F_FUNC(Name,…)` : ifx exports lower-case symbols such as `init_`, but f2py uses the source spelling `Init`.	`gen/aces2pymodule.c` `gen/aces2py-f2pywrappers.f` `f2py_gen.log`
2. Compile C	`mpicc -DNDEBUG -fPIC -O2 -I. -I$NUMPY_INC -I$PY_INC -I$F2PY_SRC` on the module C file and `fortranobject.c`.	`build/aces2pymodule.o`, `build/fortranobject.o`
3. Compile Fortran	Compile `$SOURCES` and the f2py wrapper with the `FFLAGS` below. Then `objcopy --redefine-sym props_=pyaces_vprops_props_ build/runee.o`, so Runee calls the renamed vprops PROPS.	`build/*.o`
4. Link	Build LIBDIRS/LIBNAMES, build the patched archives for colliding modules (chapter 5), then `ifx -shared -nofor-main`.	`build/aces2py.cpython-312-x86_64-linux-gnu.so`
5. Import test	`python3 -c "import aces2py; print('IMPORT OK')"`. A failure fails the build (`set -e`).	none
6. Link audit	`python3 tools/collision_audit.py || true`. Advisory only; it never fails the build.	`@COLLISION_AUDIT:` lines in the log

Fortran flags

```
FFLAGS="-diag-disable=10448 -i8 -fPIC -fp-model strict -O1 -assume minus0 -qopenmp \
        -D__fortran -D__fortran77 -D_INTEL -D_RECL_IS_WORDS_ \
        -I$ACES/include -I$ACES/acescore/include -I. -FI"
```

- `-i8`: 8-byte default INTEGER, matching aces2 and MKL ILP64.
- `-FI`: fixed-form source. `-D__fortran77` selects the F77 branch of aces2's headers.
- Headers come from aces2's `include` and `acescore/include`. `pyaces/include/` is not used.

Link command

```
ifx -shared -nofor-main -o build/aces2py.cpython-312-x86_64-linux-gnu.so build/*.o \
    -L$PATCHED_LIBDIR $LIBDIRS -Wl,--start-group $LIBNAMES -Wl,--end-group \
    -L$MKL -Wl,--start-group -lmkl_intel_ilp64 -lmkl_lapack95_ilp64 \
           -lmkl_sequential -lmkl_core -Wl,--end-group -lpthread
```

- `LIBDIRS` / `LIBNAMES` are regenerated every build from `find $ACES -maxdepth 2 -iname "*.a"` (sorted, de-duplicated). All aces2 archives sit in one group, so circular dependencies resolve.
- `-L$PATCHED_LIBDIR` comes first, so a patched copy wins over the original archive of the same name.
- `-nofor-main` stops ifx from requiring its own main program.

5. Linking into one .so: symbol collisions and their fixes

The classic ACES II build links each module into its own executable. Routines that **share a name but not an implementation**, such as vcc's `FINISH` and lambda's `FINISH`, were therefore harmless. pyaces puts all of them into one .so, so the linker picks one copy and hands it to every caller, without a warning. The result is a segfault, `@GETLST: List (x,y) does not exist`, or a **silently wrong energy**. The pyaces link step has four safeguards against this.

5.1 Localizing MAIN__

Twelve modules (`hbar joda lambda pccd props vcc vee vmol vmol2ja vscf vtran intprc`, `MAIN_COLLISION_MODULES`) contain a Fortran PROGRAM, so each defines `MAIN__`. The standalone executables need it, but in one .so it means "multiple definition of MAIN__". The build extracts each of these archives with `ar x` into a temporary directory, applies `objcopy --localize-symbol=MAIN__` to the objects that define it, and repacks them as `build/patched_libs/lib<mod>.a`. aces2's own `.a` files are not modified.

5.2 `EXTRA_LOCALIZE_SYMBOLS`: renaming the losing copy

A list of `module:symbol` pairs (around line 411). For each pair the build applies `objcopy --redefine-sym sym=pyaces_<mod>_sym` to **every object** in that module's archive. The definition and the module's own calls are renamed together, so the losing module keeps using its own implementation under the new name, and the original name is left to the winner.

> **Rule**: a pair only takes effect if its module is in `MAIN_COLLISION_MODULES` or `EXTRA_LOCALIZE_MODULES`. Pairs for modules that are not processed are **silently ignored**, and `objcopy` errors are discarded by `2>/dev/null || true`.

The build used `--localize-symbol` at first, but that sent the losing module's own calls to the winner (lambda's `vlamcc` ended up calling vcc's FINISH), so on 2026-08-11 it switched to `--redefine-sym`.

Main collisions resolved so far

Symbol	Renamed (losing) copy	Symptom / note
`FINISH`	lambda, molcas	Runcc reported the iteration-0 TOTENERG instead of the converged one
`GSCHMIDT`	joda, libr, vcceh, vea	SIGSEGV from an argument-count mismatch
`MODF`	lambda, fsip, vcceh, hcmult, vea, lcct	Failures on EOM paths
`EXPDEN`, `PROPS` and vprops helpers	props, vcceh / vprops	Dipole calculation in Runee
`HBARXC`, `DT2INT2`, `DRAOLAD`, `T2TOAO`, `GETAOINF`, `Z2TOMO`	vcceh, lcct, hcmult, lambda, vee, dens	GETLST "list does not exist"; Z2TOMO gave an error of about 1e-5
`W45T3DRV`	lambda	lambda's 8-argument copy won over trp's 4-argument one
`CMPENG`	rpa	Singles term dropped: a silent 2.49e-05 error

`XGEMM`	tools	Illegal argument passed to DGEMM
`CHECK_T2`, `INITIN/INITSN/DIISLST/DODIIS0`	ringcc / lambda	vcc used other modules' DIIS and initialisation routines
`ABCD` (COMMON shadow)	intgrt	Every KS case faulted on instruction fetch (section 5.4)
`getdt1`, `fndmaxs`, `t2l2rng`, `r1l2y2c`	vcc, vee, lcct, lcct	The winner depended on link order (make 4.2 vs 4.3); pinned in eab3c4e6

The cause and diagnosis of each entry are recorded in the comments of `build.sh` (roughly lines 118–410).

5.3 `NEWLY_LINKED_MODULES`: bulk handling of re-enabled modules

`a2mix aux rpa scrnc` were `.skip` directories until they were re-enabled, so the classic build never linked them next to the other modules. The build collects the `T` symbols of every `aces2/lib/lib*.a` and automatically adds to `EXTRA_LOCALIZE_SYMBOLS` every symbol (except `MAIN__`) that one of these four modules shares with another module. The log shows `shadowed by <mod>, renaming: …`.

5.4 COMMON blocks that shadow subroutines

A Fortran `COMMON /abcd/` emits a tentative symbol `abcd_` (`C` in `nm`). A subroutine `ABCD` is also `abcd_`. If the COMMON satisfies the reference before the linker extracts the object holding the subroutine, `call abcd` jumps into zero-filled BSS. The symptom is a SIGSEGV on instruction fetch, recognisable in gdb by `si_addr == $pc`.

- **rename-definer**: if only the defining module calls the routine, add `definer:sym` to `EXTRA_LOCALIZE_SYMBOLS` (e.g. `intgrt:abcd_`).
- **rename-commons**: if other modules call it too, renaming would orphan them, so the COMMON block has to move instead.

Nine latent cases (mopac `deriv/grid/local/polar`, nddo `guess/local`, aces2 and ccd_light `energy`, pdens `one/tab`, vdint `sym2`) were renamed ahead of time.

5.5 Keeping patched_libs clean

`build/patched_libs` comes before aces2/lib on the `-L` path, so a stale file there shadows the real archive. A `libintgrt.a` left over from an experiment once caused `undefined symbol: intgrt_run_`. The directory is now removed and rebuilt on every build.

5.6 Finding out which copy won

```
# First choice: ask the linker (add to the link command)
-Wl,-trace-symbol=getdt1_
# Fallback: match the size in the .so against the archives
nm -S Main/build/aces2py*.so | grep ' getdt1_$'
nm -S ../aces2/lib/libvcc.a ../aces2/lib/libvee.a | grep ' getdt1_$'
# COMMON shadow check: B or D instead of T means data, not code
nm Main/build/aces2py*.so | grep -E ' (abcd_|hbar_)$'
# Which symbol does a pyaces object actually call?
nm Main/build/runhbar.o | grep -i hbar
```

6. The link-resolution audit (`collision_audit.py`)

It runs automatically as build step 6 and can also be run by hand. It needs an already built `Main/build/aces2py*.so`.

```
python3 tools/collision_audit.py            # report against the baselines
python3 tools/collision_audit.py --update   # overwrite both baseline files with the curren
t findings
```

6.1 What it checks

(1) Resolution differences. For every symbol defined by more than one module:

1. Reconstruct the classic link order from `ACES_LINK_LISTS` in `aces2/Makefiles/GNUmakefile`.
2. Identify the copy actually in the .so by its `nm -S` size (skipped if sizes cannot tell the copies apart).
3. For each calling module, compare it with the copy the classic build would have given (its own copy, otherwise the first library in its link list that defines the symbol).
4. If the two differ, and their comment-stripped sources differ too, report `symbol winner classic caller`.

Pairs already renamed by `build.sh` are excluded; the audit reads `EXTRA_LOCALIZE_SYMBOLS` and `NEWLY_LINKED_MODULES` straight from build.sh.

(2) COMMON shadows. It reports symbols that are `T` in one archive and `C` in another and B/b/D/d in the .so, with a verdict of `rename-definer` or `rename-commons`.

6.2 Reading the output

```
@COLLISION_AUDIT: 0 subroutines shadowed by a same-named COMMON block ...
@COLLISION_AUDIT: 296 resolution differences (296 in the baseline)
  resolved since the baseline: ...             # in the baseline, now gone
@COLLISION_AUDIT: NEW resolution differences ...
    getdt1_   .so gives vcc, lcct expects lcct   # new: needs action
```

6.3 When NEW entries appear

1. Compare the two implementations (argument count, list numbers, what they compute). If they compute the same thing, the entry can be accepted into the baseline.
2. If they differ, choose the copy to rename and add `"<loser>:<symbol>"` to `EXTRA_LOCALIZE_SYMBOLS`. Make sure its module is in `EXTRA_LOCALIZE_MODULES`.
3. Rebuild pyaces and confirm the NEW entry is gone and no other symbol flipped. Adding a module to the processed set can change how its other symbols resolve.
4. Update the baseline with `--update` only after the suite has passed.

> **The baseline is not a to-do list**: its 296 entries are known, accepted differences. Renaming 42 "safe-looking" entries in one pass once dropped the suite from 45 to 31 PASS. Also, `ACES_LINK_LISTS` only covers modules that build executables, so callers inside library modules (libr, libr2, tools, acescore, ...) are invisible to the audit. A call bound to a COMMON block with no `T` definition anywhere (the `hbar_` case in chapter 11) is invisible too.

7. The `aces2py` API

7.1 Signatures come from the Fortran sources

Given both `aces2.pyf` and `PYF_SOURCES`, f2py builds the module twice: once from the `.pyf` (lower-case names, with COMMON blocks) and once from scanning the sources (source spelling). Both write the same `aces2pymodule.c`, so **the second one wins** (visible in `f2py_gen.log`). As a result:

- Python names follow the capitalisation in the source: `Init`, `Dgetrecpy`, `IGetrecpy` (capital G).
- Argument types and intents come from the source. Intents in the `.pyf` are ignored; to change one, put a `Cf2py intent(...)` line in the `.F` file. Every `Run*` takes an `Iuhf` argument.
- The COMMON blocks declared in the `.pyf` (`/flags/`, `/jobarc/`, `/lists/`, ...) are not exposed.
- Check exact signatures with `print(aces2py.Runcc.__doc__)` or the docstrings in `gen/aces2pymodule.c`.

7.2 Array arguments

Every function is a Fortran subroutine. Apart from the two functions in 7.4 that return a value, results are **written into numpy arrays the caller passes in**. f2py wraps those arrays as "input", so if the dtype or memory layout is wrong it silently passes a **copy**, and whatever Fortran writes is lost with the copy.

> **Always** pre-allocate C-contiguous 1-D arrays: `np.zeros(n, dtype=np.float64)` for reals, `np.zeros(n, dtype=np.int64)` for integers. Record names are 8 characters, padded with blanks (`"IFLAGS  "`).

7.3 Calculation drivers

`Iuhf`: 0 = RHF, 1 = UHF/ROHF (`Integer*8` on the Fortran side, `long long` in C). No driver calls `aces_fin`, so the session stays open afterwards and records can be read.

Function	Source	What it runs
`Init()`	`Main/init.F`	`aces2_main`: `Joda_` (parse ZMAT, create JOBARC), `Aces_ja_fin`, `aces_init` (allocate core, set up flags and lists, reopen JOBARC). **Call it first, before every driver except Runee.**
`Runints(Iuhf)`	`runints.F`	`Ints_` (vmol AO integrals) and `V2ja`
`Runscf(Iuhf)`	`runscf.F`	Integrals first if there is no IIII file, then `Vscf`
`Runcc(Iuhf)`	`runcc.F`	SCF first if it has not run (no `SCFEVCA0` record), then `aces_init_chemsys`, `Vtran`, `Init_legacies`, `Intproc`, `Vcc`. Covers the MBPT/CC family.
`Runhbar(Iuhf)`, `Buildhbar()`	`runhbar.F`, `buildhbar.F`	The Runcc steps, then `Hbar_`. If SCF is done (`SCFEVCA0`) it starts at Vtran; if CC is done (`CC_DONE`) it runs only Hbar. Buildhbar reads iuhf from `IFLAGS(11)`. **Do not call it after Runcc**: it takes the SCF-done branch and runs Vtran a second time.
`Runprops(Iuhf)`	`runprops.F`	`Props_`
`Runpccd(Iuhf)`	`runpccd.F`	Self-contained: Joda, Ints, aces_init, V2ja, Vscf, Vtran, Intproc, Vcc, Vlambda, Pccd

`Runee(Iuhf)`	`runee.F`	Self-contained EOM-CC: Joda, aces_init, Ints, V2ja, vprops Props (renamed copy), Vscf, Vtran, Intproc, Vcc, Vlambda, Vee. **Do not call `Init()` before it.**
`Rundirmp2(Iuhf)`	`rundirmp2.F`	SCF if needed, then `Dirmp2_run(0)`. Iuhf is not forwarded.
`Runintgrt(Iuhf)`	`runintgrt.F`	SCF if needed, then `Intgrt_run` (KS/DFT numerical integration, XC energy)
`Acesjafin()` / `Acesjainit()`	`acesjafin.F` / `acesjainit.F`	Close (flush) / reopen the JOBARC session. Reopening an already open session is a hard `aces_exit`.

7.4 JOBARC records

Signature	Description
`Dgetrecpy(Name, Dest, [Length])`	Read a float64 record into `Dest`, e.g. `"TOTENERG"`, `"SCFENEG "`
`IGetrecpy(Name, IDest, [Length])`	Read an int64 record, e.g. `"IFLAGS "` (100), `"NBASTOT "`
`Dputrecpy(Name, Source, [Length])`	Write a float64 record
`Iputrecpy(Name, Isource, [Length])`	Write an int64 record
`Cputrecpy(Name, Length, Source)`	Write a character record
`Dest = Cgetrecpy(Name, Length)`	Read `Length` bytes of a character record and return them as `bytes` (length 1024, blank-padded). A length above 1024 is an `aces_exit`.
`Length = Getreclenpy(Name)`	Return the record length in integer words, or `-1` if the record does not exist.

Lengths are in integer words; with `-i8`, one integer = one real = 8 bytes. If `Length` is omitted, the array length is used.

7.5 MO lists (MOIO)

Signature	Description
`Getlstpy(Column_Index, No_columns, Irrep, Left_index, Right_index, Destn, Ndim)`	Calls `A2_getlst(Destn, Column_Index, No_columns, 1, Left_index, Right_index)`. In practice **`Left_index` is the irrep and `Right_index` the list number**; `Irrep` and `Ndim` are ignored.
`Putlstpy(Source, Column_Index, No_columns, Irrep, Left_index, Right_index, [Ndim])`	The write counterpart of Getlstpy
`Getallpy(Destn, Irrep, Right_index, [Ndim])`	Read all irrep blocks of list `Right_index` with total symmetry `Irrep`
`Putallpy(Source, Irrep, Right_index, [Ndim])`	The write counterpart

A draft table of list numbers is in `Python/moio_len.py` (e.g. T2(Ij,Ab) = 46, T1(AI) = 901, W<AB||IJ> = 14). It is not valid Python; use it as a reference only.

7.6 AO integrals

Signature	Description
`Get1ehpy(Name, Dest, Ndim)`	Read `"overlap"`, `"oneh"` or `"kinetic"` from the IIII file and expand it into a full `Ndim×Ndim` matrix (`Ndim = NBASTOT`).
`Get2ehpy(Name, Dest, Ndim)`	`"2elints"`: expand all two-electron integrals from IIII/IJIJ/IIJJ/IJKL into an array of size $\texttt{Ndim}^4$. Mind the memory.

8. Running calculations

8.1 Runtime environment

- **Working directory**: must contain `ZMAT` and `GENBAS`. ACES writes JOBARC, JAINDX, IIII/IJIJ/IIJJ/IJKL, MOINTS, MOABCD, MOL, NEWMOS and others into the same directory. Memory size comes from the ZMAT keywords.
- **Libraries**: the Intel runtime and MKL are needed. Load the same module set as for the build, or set `LD_LIBRARY_PATH`. Otherwise the import fails with `libmkl_intel_ilp64.so.2: cannot open shared object file`.
- **Import path**: put `pyaces/Main/build` on `sys.path` or `PYTHONPATH`.

8.2 Basic usage

```
import os, sys, numpy as np
os.chdir("/path/to/workdir")                # contains ZMAT and GENBAS
sys.path.insert(0, "/path/to/ACES_PROGRAM/pyaces/Main/build")
import aces2py

aces2py.Init()                              # skip for Runee
aces2py.Runcc(0)                            # 0 = RHF, 1 = UHF/ROHF

e = np.zeros(1, dtype=np.float64)
aces2py.Dgetrecpy("TOTENERG", e)
print("E =", e[0])
print("TOTENERG length:", aces2py.Getreclenpy("TOTENERG"))
```

8.3 Reading and modifying records

```
iflags = np.zeros(100, dtype=np.int64)
aces2py.IGetrecpy("IFLAGS  ", iflags)
iflags[1] = 1                               # IFLAGS(2) = CALC
aces2py.Iputrecpy("IFLAGS  ", iflags)
aces2py.Acesjafin()                         # flush before an external executable reads JO
BARC
# ... run ../aces2/bin/xvtran etc. as subprocesses ...
aces2py.Acesjainit()
```

8.4 Rules

- **One calculation per process.** Do not run several ZMATs in a row in one process. The test runner starts a new process for each case.
- **Do not run the same module twice.** Calling Vtran/Vcc twice in one process has converged to wrong values. This includes calling Runhbar after Runcc.
- **Runee and Runpccd call Joda themselves.** Do not call `Init()` before Runee.
- **Out of scope**: geometry optimisation, frequencies, analytic derivatives (gradients and Hessians), Brueckner, `HFSTABILITY=FOLLOW` and CCSD(TQ) cannot run under pyaces yet (`rungrads.F` is excluded from the build).

8.5 FNO special case (`pyaces_run_case.py`)

When ZMAT contains `FNO_KEEP=`, the classic driver runs a preliminary Vtran/Intproc/Vcc/Fno pass to choose which virtual orbitals to freeze. Doing that in-process would call Vtran/Vcc twice, so the runner does this instead:

1. Run `Runscf`.
2. Temporarily change `IFLAGS` 2, 93, 83 and 100, and move `NUMDROPA` to `FNOFREEZ`.
3. Call `Acesjafin()`, then run the standalone `../aces2/bin/{xvtran,xintprc,xvcc,xfno}` as subprocesses (logs: `fno_prelim_<exe>.log`).
4. Restore the flags; regenerate IIII with `xvmol` / `xvmol2ja` if it was consumed.
5. Call `Acesjainit()`, then `Runcc`.

8.6 `pyaces_run_case.py` output protocol

```
python3 -u pyaces_run_case.py <workdir> <entrypoint> <iuhf> <record>
# progress: PYACES_CASE: ...
# success:  PYACES_CASE_VALUE: <repr>   /   PYACES_CASE_RESULT: SUCCESS <%.10f>
# failure (exit 1): IMPORT_FAILED, <ENTRY>_FAILED, RECORD_ZERO_OR_MISSING, DGETRECPY_FAILED
```

A record value of exactly 0.0 is treated as missing.

9. The test suite

9.1 Running it

```
./run_tests.sh pyaces                                    # top level (log: build_logs/unifie
d_test_pyaces.log)
cd pyaces && python3 pyaces_suite_runner.py              # directly
cd pyaces && python3 pyaces_suite_runner.py 010a 117b    # selected cases only
```

- Case input: `aces2/tests/fast/cases/<name>/ZMAT`; basis: `aces2/tests/fast/GENBAS`. There is no pyaces-specific test directory.
- Working directory: `$PYACES_RUN_DIR/<name>`. Off HiPerGator, `run_tests.sh` sets it to `build_logs/pyaces_suite_run`; the runner's own default is HiPerGator's `/blue/...` path.
- Each case gets a fresh directory with ZMAT copied in and GENBAS symlinked, then `pyaces_run_case.py` runs with a 300-second limit. Log: `<dir>/run.log`; summary: `$RUN_DIR/summary.tsv`.
- Local workstation: `OMP_NUM_THREADS=1 MKL_NUM_THREADS=1 taskset -c 20-23 flock -s /tmp/qtoy-bench.lock ./run_tests.sh pyaces`. ccsd server: `qsub build_logs/test_ccsd.pbs`.

9.2 Grading

Status	Condition
PASS	Exit code 0 and $\lvert\text{computed} - \text{reference}\rvert \le 1\times10^{-6}\ E_h$ (`TOLERANCE`, no per-case overrides)
MISMATCH	Exit code 0, but the difference exceeds the tolerance
NO_RESULT	Exit code 0, but no `PYACES_CASE_RESULT: SUCCESS` line
TIMEOUT	More than 300 seconds
FAIL_CLEAN	Positive exit code with a recognised message: an `@ROUTINE:` banner, "Assertion failed", "did not converge", "forrtl: severe", ...
CRASH	Killed by a signal (negative code), or a positive exit with no recognised message

Latest result: 51/51 PASS, largest difference 4.95×10^{-8} (case 078).

9.3 `suite_classification.tsv`

Columns: `name`, `category`, `reason`, `entrypoint`, `iuhf`, `record` (the JOBARC record compared, `TOTENERG` or `SCFENEG`), `ref_value` (Fortran E/D format). The runner only runs rows whose category is SCF, CC, EOM, PCCD or KS.

Category	Count	Entry point / iuhf
CC	31	Runcc (iuhf 0: 12, 1: 16), Rundirmp2 (3)
EOM	8	Runee (0: 6, 1: 2)
SCF	9	Runscf (0: 6, 1: 3)
KS	3	Runintgrt (0)
PCCD	0	none

EXCLUDED	59	Geometry optimisation, VIB, derivatives, MRCC, Brueckner, HFSTABILITY=FOLLOW, CCSD(TQ), …
SCRIPT	15	Cases driven by scripts rather than a ZMAT

9.4 Do not rerun `classify_suite.py`

`classify_suite.py` first built the TSV from `aces2/tests/fast/test_results` and the ZMATs. Several rows were later edited **by hand** and would be lost if the script were rerun: the KS → Runintgrt rows, 117a/b/c → Rundirmp2, and the exclusions of 020 (CCSD(TQ)), the 022 family (Brueckner), 083 and 124d.fno (HFSTABILITY=FOLLOW). Edit the TSV directly, or move those exceptions into `classify()` before regenerating it.

> **Gap in the roll-up**: the pyaces summary in `run_tests.sh` only counts `PASS:`, `FAIL_CLEAN:`, `CRASH:` and `NO_RESULT:`. MISMATCH and TIMEOUT are not counted in TOTAL, so if the total is not 51, read the Summary block in `unified_test_pyaces.log` yourself.

10. Extending pyaces: new entry points and new tests

10.1 Exposing a new Fortran routine to Python

1. **Make the aces2 side callable.** If the target is a PROGRAM, split its body into a subroutine (the "wrapper-split" pattern used for `dirmp2_run`, `intgrt_run`, `joda_`, `hbar_`, `props_`). Rebuild aces2 first.
2. **Write `a2_pyf/<name>.F`.**
   - Fixed form, with every `#include` starting in column 1 (oneAPI 2023.2 ifort rejects indented includes).
   - Declare every dummy argument **explicitly** as `Integer*8` or `Double Precision`. A plain `Integer` makes f2py pass a 4-byte C `int` while `-i8` Fortran reads 8 bytes.
   - Mark outputs with `Cf2py intent(out) <arg>`; the `.pyf` intents are ignored (7.1).
   - The capitalisation of the subroutine name becomes the Python name.
   - Use `Icore(i0), Icrsiz` from `icore.com`/`istart.com` for core memory (see `runcc.F`). Do not call `aces_fin`.
   - Call aces2 routines by their real symbol: if aces2 defines `SUBROUTINE HBAR_`, the call must be `Call Hbar_`; `Call Hbar` would bind to a COMMON block (chapter 11).
3. **Add a block to the interface in `Main/aces2.pyf`** (for documentation; the source decides the real signature).
4. **Add the file to `PYF_SOURCES` in `build.sh`.** The `a2_pyf/*.F` glob compiles it automatically unless it is on the exclusion list.
5. **Handle collisions.**
   - If a newly pulled-in module has a PROGRAM, add it to `MAIN_COLLISION_MODULES`.
   - For a same-named routine, add `loser:sym_` to `EXTRA_LOCALIZE_SYMBOLS` and the module to `EXTRA_LOCALIZE_MODULES`.
   - If a pyaces object calls a renamed routine directly, add an `objcopy --redefine-sym` after step 3 (as done for `runee.o`).
   - If you re-enable a `.skip` module, add it to `NEWLY_LINKED_MODULES`.
6. **Build and check**: "IMPORT OK", the new wrapper in the second (source-case) module build in `f2py_gen.log`, and `print(aces2py.<Name>.__doc__)`.
7. **Handle NEW audit entries** as described in section 6.3.
8. **Check that every name you call is code**: `nm Main/build/<name>.o` lists the undefined symbols your object calls, and `nm Main/build/aces2py*.so | grep ' <sym>$'` must show `T`. `B` or `D` means the call is bound to a COMMON block.
9. **Add a test case** and run the whole suite.

10.2 Adding a test case

1. Add the case to the aces2 suite: `aces2/tests/fast/cases/<name>/ZMAT` and a three-line entry in `aces2/tests/fast/test_results` (`name src` / `RECORD 1` / reference value).
2. Add a row to `pyaces/suite_classification.tsv` by hand (category, entrypoint, iuhf, record, ref_value).

3. Check it with `PYACES_RUN_DIR=... python3 pyaces_suite_runner.py <name>`, then read `<dir>/run.log` and `summary.tsv`.
4. Make sure the full suites (`./run_tests.sh aces2 pyaces`) still pass.

10.3 Checklist after a change

What changed	What to verify
Fortran in a2_pyf / Main	pyaces build, IMPORT OK, suite 51/51, `nm` check of the changed objects
Rename lists in build.sh	Audit (no NEW entries; any "resolved" entries are intended), suite, then `--update`
aces2 sources or flags	`git clean -fdXq aces2`, rebuild aces2 and pyaces, aces2 suite 125/125, pyaces 51/51, audit
Toolchain / another machine	All of the above, plus identical audit results on both machines

11. Known issues (with fix history)

> **Fixed on 2026-10-23 (commit f97a9759)**: three defects found while writing this manual.
>
> - **`Call Hbar` in Runhbar/Buildhbar**: the symbol `hbar_` is not a subroutine but lambda's `COMMON /HBAR/` (`B hbar_` in the .so), so the SCF-done and CC-done branches could jump into BSS. Changed to `Call Hbar_` (= `hbar__`). The audit cannot catch this case because there is no `T hbar_` anywhere, which is why step 8 of section 10.1 asks for an `nm` check. Verified with `Init → Runscf → Runhbar` on case 026 (CCSD energy matches the reference to 7×10^{-11}).
> - **Width of `Iuhf`**: the `Integer Iuhf` in `run*.F` made f2py pass a 4-byte C `int` while `-i8` Fortran read 8 bytes. Now declared `Integer*8 Iuhf`, so both sides are 64-bit.
> - **`Getreclenpy`, `Cgetrecpy`**: their outputs never reached Python. `Cf2py intent(out)` now makes them return the length and the string (section 7.4); the debug prints in Cgetrecpy were removed.

The remaining issues below are present in the current code. They do not affect the paths the suite exercises today, but you will meet them when using or extending the affected features.

#	Issue	Details / recommended action
1	Meaning of `Getlstpy` arguments	`Irrep` and `Ndim` are ignored; `Left_index` is used as the irrep.
2	The `.pyf` is effectively ignored	COMMON-block exports and intents in it are not applied (section 7.1).
3	Python version is fixed	The output name is hard-coded to `cpython-312`. Other versions would need `sysconfig.get_config_var('EXT_SUFFIX')`.
4	Header copies differ	The nine `Main/*.com` files differ from aces2's headers (explicit `integer*8`, no include guards). Sizes match thanks to `-i8`, so this is harmless today, but if aces2's headers change, init.F and the rest would see different COMMON layouts.
5	Runpccd and Init	Runpccd calls Joda itself, yet the runner calls `Init()` before it. Untested, since there are no PCCD cases.
6	Errors are ignored	Typos in `EXTRA_LOCALIZE_SYMBOLS` and pairs for unprocessed modules are silently ignored.
7	ifort check in the top-level build	The top-level `build.sh` requires `ifort` before the pyaces shim is on PATH, so an ifx-only toolchain is refused.
8	Assumes environment modules	`pyaces/build.sh` always runs `module purge/load`, so it fails in setvars/path mode or on hosts without a `module` command.
9	Test roll-up and classifier	See sections 9.3–9.4.
10	Symmetry-block offset in `Get1ehpy`	`Index = Index + Nelms*(K-1)` at `get1ehpy.F:101` looks wrong. Verify the result on a molecule with symmetry before relying on it.

12. Development history and server deployment

12.1 History

Period	Work
2023-11	Original acespy (Ajith Perera), built as per-module shared libraries (`Fxx/`) wrapped with f2py.
2026-08-08 – 08-11	Subtree-merged into ACES_PROGRAM (`ff876bec`). Hard-coded paths removed and the link list generated from the aces2 archives (`37d57822`). Fixing the first collision (FINISH) took the suite from 13 to 30 of 58 PASS. EOM-path collisions fixed (GSCHMIDT, MODF, Z2TOMO, ...); switched from localize to redefine-sym.
2026-09-02 – 09-04	Portable test runners; W45T3DRV, CMPENG, XGEMM, CHECK_T2 and INITIN collisions fixed; bulk renaming via NEWLY_LINKED_MODULES; the audit tool; the Rundirmp2 and Runintgrt entry points; exclusions cleaned up.
2026-09-04 – 09-11	COMMON shadowing found (ABCD → the KS crash); the KS category brought the suite to 51/51; COMMON audit added; nine latent shadows renamed pre-emptively.
2026-10-14	Ported to a second machine (ccsd): init.F includes in column 1 (`73944a6d`); aces2 built with `-assume dummy_aliases` (`dbed9377` , ifx 2023.2 miscompiled vgamma2); sorted source lists, `LC_ALL=C` and extra renames (`eab3c4e6`). 51/51 and an identical 296-entry audit on both machines.
2026-10-23	Defects found while writing this manual fixed (`f97a9759`): `Hbar` → `Hbar_` in Runhbar/Buildhbar, `Iuhf` as `Integer*8` , Getreclenpy and Cgetrecpy return their values. Still 51/51 on both machines.

12.2 Deployment on the ccsd server

- Installed at `ycpark@172.24.7.130:/home/ycpark/program/ACES` . The server has no internet access, so it is updated from the workstation with `git push ccsd master` (the server repository has `receive.denyCurrentBranch=updateInstead`).
- Toolchain: `intel/oneapi_2023.2.0` (ifx 2023.2, MKL 2023.2, Intel MPI 2021.10) and `python/3.12.3` .
- Build and test only through PBS: `build_logs/build_ccsd.pbs` (`qsub -v TARGETS=pyaces …`), `build_logs/test_ccsd.pbs` , and `build_logs/rebuild_test.pbs` (clean aces2 rebuild plus tests).
- Built with `-march=core-avx2` , so the binaries run on any of the ccsd50/51/52 nodes.

12.3 Lessons from the toolchain differences

- After a compiler change, rerun all three suites (aces2, aces3, pyaces). ifx 2023.2 handled two pieces of code differently from 2024.0, which had passed them.
- Compare the link-audit results across machines. The same sources linked different implementations because of make and locale differences.

Appendix A. Legacy files

Path	What it is
`Fxx/`	A copy of aces2 sources from the 2023 acespy per-module shared-library build (45 directories, about 6,100 files). It has drifted considerably from today's aces2 and no script references it.
`include/`, `a2_pyf/include_no/`	Old header copies. The build uses aces2's include directories.
`Main/Archive/`, `Main/compile_acespy`	Early prototypes and the old `f2py -c` build recipe.
`a2_pyf/Archive/`	Files and test code from when each routine had its own `.pyf`.
`Main/Pilot_tests/`	Early COMMON/icore experiments.
`Exp/`	A standalone f2py tutorial (`add.F`, `check_strngs.F`, ...).
`Python/moio_len.py`	Draft reference table of MO list numbers (contains syntax errors).
`rename.py`	Renames `.a` to `.so` under a hard-coded path.
`README`, `frequently_asked_questions`	Old build notes and the generic f2py FAQ.
`suite_classification.tsv.before_*`	TSV snapshots taken before classification fixes.
`LICENSE`	GPL v3, **still in force**.

Appendix B. Command summary

```
# Build (local workstation, inside the allocated cores)
taskset -c 20-23 flock -s /tmp/qtoy-bench.lock ./build.sh aces2 pyaces

# Tests
OMP_NUM_THREADS=1 MKL_NUM_THREADS=1 \
  taskset -c 20-23 flock -s /tmp/qtoy-bench.lock ./run_tests.sh aces2 pyaces
cd pyaces && python3 pyaces_suite_runner.py 010a       # one case

# Link audit
cd pyaces && python3 tools/collision_audit.py
cd pyaces && python3 tools/collision_audit.py --update # only after verification

# Inspect the module
python3 -c "import sys; sys.path.insert(0,'pyaces/Main/build'); import aces2py; print(aces2
py.__doc__)"
nm pyaces/Main/build/aces2py*.so | grep ' runcc_$'

# Clean rebuild after changing aces2 flags
git clean -fdXq aces2 && ./build.sh aces2 pyaces

# ccsd server
git push ccsd master
ssh ycpark@172.24.7.130 'cd program/ACES/build_logs && qsub rebuild_test.pbs'
```